

FECAP - FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação

6º Semestre - Turma 6º CCOMP

SMASH

**RELATÓRIO DE VULNERABILIDADES E PLANO DE
SEGURANÇA DO SISTEMA**

Entrega 1 - Redes de Computadores e Cibersegurança

Projeto 7 - Grupo Smash

Felipe Vallim Soares - 24026060

Guilhermy Mariano Lisboa Garcia - 23025371

Gustavo Oliveira Demetrio - 24026213

Saulo Pereira de Jesus - 24026095

Prof. Victor Bruno Alexander Rosetti

São Paulo

2026

APRESENTAÇÃO DO PROJETO

O Smash é uma parede de treino de tênis formada por nove alvos com sensores piezoelétricos. O ESP32 identifica o alvo atingido e envia o impacto ao aplicativo, que conduz os modos de jogo. A API registra contas, progressão e sessões no PostgreSQL. Como esses componentes se comunicam por Wi-Fi, a segurança influencia tanto a proteção dos dados quanto a continuidade do treino.

A análise encontrou uma boa base no código, com proteção de senhas, autenticação, validação de entradas e filtragem dos eventos da parede. Os principais riscos estão nas conexões HTTP e WebSocket sem criptografia, na senha previsível do ponto de acesso do ESP32 e na confiança excessiva nos resultados enviados pelo aplicativo.

1 OBJETIVO E ESCOPO

Este relatório identifica vulnerabilidades do sistema e do código, registra as proteções já aplicadas e recomenda correções proporcionais ao protótipo acadêmico. O escopo cobre o firmware do ESP32, o aplicativo React Native com Expo, a API NestJS e o PostgreSQL no estado atual da Entrega 1.

A revisão foi estática: foram examinados configuração, autenticação, persistência, protocolo da parede e dependências de produção. Não foram realizados ataques em rede real nem testes destrutivos no ESP32. O MQTT é citado apenas como risco da integração futura, pois ainda não está implementado.

A análise corresponde ao estado do repositório <https://github.com/2026-2-NCC6/Projeto7> no commit [1a99fd6d2bd2312928ef4f3add32b903e67b3452](https://github.com/2026-2-NCC6/Projeto7/commit/1a99fd6d2bd2312928ef4f3add32b903e67b3452), sem alterações locais pendentes em *src/Entrega1*. Os caminhos de arquivo citados no item 5 são relativos a essa pasta. O código-fonte correspondente acompanha este relatório na entrega, de modo que cada item descrito a seguir possa ser conferido na mesma versão do sistema que foi analisada.

2 ARQUITETURA E MÉTODO DE ANÁLISE

O fluxo principal é: alvo físico → sensor → ESP32 → WebSocket → aplicativo → HTTP → API → PostgreSQL. Cada ligação é uma fronteira de confiança, porque recebe dados ou credenciais de outro componente.

Quadro 1 - Componentes, dados e fronteiras de confiança.

Componente	Dados tratados	Fronteira de segurança
ESP32	Alvo, intensidade, tempo e sessão.	Wi-Fi, comandos WebSocket e disponibilidade do dispositivo.
Aplicativo	Conta, token, sessão e pontuação.	Mensagens da parede e dados enviados à API.
API	Autenticação, progressão e sessões.	Validação, autorização e limitação de requisições.
PostgreSQL	E-mail, hash de senha e histórico.	Credencial, acesso de rede e integridade dos dados.
MQTT futuro	Impactos publicados por parede.	Autenticação, TLS e autorização por tópico.

Os riscos foram classificados por probabilidade e impacto em níveis baixo, médio ou alto. A prioridade considera a combinação desses níveis e os efeitos sobre confidencialidade, integridade e disponibilidade. “Atual” indica uma condição presente; “planejado” indica um controle necessário para uma integração ainda não implementada.

3 SEGURANÇA JÁ APLICADA

O projeto já possui controles relevantes que devem ser preservados nas próximas entregas:

Quadro 2 - Principais controles presentes no projeto.

Controle	Aplicação no Smash	Benefício
bcrypt e login genérico	A senha vira hash com custo 10 e erros não revelam se a conta existe.	Reduz exposição de senha e enumeração de usuários.
JWT e segredo obrigatório	Rotas pessoais exigem token; a API não inicia sem segredo configurado.	Evita acesso anônimo e assinatura com valor padrão.
Validação da API	Campos desconhecidos são rejeitados e tipos e limites são conferidos.	Reduz entradas inesperadas e dados malformados.
TypeORM e transações	Consultas parametrizadas e gravações relacionadas são atômicas.	Reduz injeção e inconsistência no banco.
Restrições e segredos	Há chaves e índices no banco; arquivos .env ficam fora do Git.	Protege integridade e reduz vazamentos acidentais.
Token somente em memória	O aplicativo não grava o token no AsyncStorage.	Diminui a persistência do token no aparelho.
Validação da parede	Eventos inválidos, duplicados ou fora de ordem são descartados.	Evita que dados defeituosos cheguem ao jogo.

Esses controles atuam em diferentes pontos do fluxo do Smash e se complementam. A autenticação protege o acesso às informações pessoais, a validação impede que entradas inesperadas avancem pela API e as restrições do banco preservam a consistência dos registros. Na comunicação com a parede, a filtragem dos eventos reduz a chance de mensagens defeituosas interferirem no funcionamento do jogo.

Mesmo com essa base, a proteção depende de os controles continuarem ativos e configurados corretamente. Segredos devem permanecer fora do repositório, as validações não devem ser removidas para facilitar testes e alterações no banco precisam manter as restrições existentes. Essa manutenção é importante porque as vulnerabilidades apresentadas a seguir estão principalmente nas conexões e nos limites entre os componentes, e não na ausência completa de segurança.

4 RESUMO DAS VULNERABILIDADES

O quadro abaixo reúne os riscos encontrados. “Crítico” exige correção antes de expor o sistema a uma rede compartilhada; “alto” deve entrar na próxima iteração; “médio” deve ser acompanhado e planejado.

Quadro 3 - Registro priorizado de vulnerabilidades.

ID	Vulnerabilidade	Prob.	Impacto	Prioridade
R01	API usa HTTP sem criptografia.	Alta	Alto	Crítico
R02	WebSocket da parede não possui criptografia nem autenticação.	Alta	Alto	Crítico
R03	SoftAP usa senha padrão previsível e versionada.	Alta	Alto	Crítico
R04	Backend aceita o resultado declarado pelo aplicativo.	Alta	Alto	Crítico
R05	Qualquer cliente WebSocket pode alterar o identificador da sessão.	Alta	Médio	Alto
R06	Login e cadastro não limitam tentativas.	Alta	Médio	Alto
R07	ESP32 não limita clientes, tamanho ou frequência de mensagens.	Média	Alto	Alto
R08	API aceita CORS amplo e escuta em todas as interfaces.	Média	Médio	Médio
R09	Banco possui credenciais padrão de desenvolvimento.	Média	Alto	Alto
R10	JWT dura sete dias e não possui revogação controlada.	Média	Médio	Médio
R11	Dependências apresentam alertas conhecidos no npm audit.	Média	Alto	Alto
R12	Eventos de segurança têm pouca trilha de auditoria.	Média	Médio	Médio
R13	MQTT futuro ainda não possui controles definidos no código.	Média	Alto	Alto

5 DETALHAMENTO DOS RISCOS

5.1 R01 COMUNICAÇÃO HTTP SEM CRIPTOGRAFIA

Diagnóstico. O aplicativo envia o token JWT à API pelo endereço HTTP configurado por padrão (*frontend/src/config/env.ts:20-24* e *backend/src/main.ts:24*). **Risco e impacto.** Alguém na mesma rede pode observar o tráfego, capturar o token ou alterar requisições, comprometendo contas e histórico. Prioridade crítica. **Correção.** Usar HTTPS por meio de um proxy com certificado confiável e manter a porta da API restrita à rede necessária.

5.2 R02 WEBSOCKET SEM CRIPTOGRAFIA E SEM AUTENTICAÇÃO

Diagnóstico. A parede transmite impactos em ws e aceita clientes sem token, certificado ou pareamento (*backend/firmware/smash_wall/wall_link.cpp:25-50* e *frontend/src/device/config.ts:4*). **Risco e impacto.** Um dispositivo próximo pode ler impactos, enviar comandos ou interromper a sessão. Prioridade crítica. **Correção.** Na demonstração, usar rede isolada e cliente único; no uso real, adotar pareamento e WSS ou um gateway seguro.

5.3 R03 SENHA PADRÃO DO SOFTAP

Diagnóstico. O firmware mantém o SSID smash-wall e a senha smash1234 no código versionado (*backend/firmware/smash_wall/config.h:29-30*). **Risco e impacto.** A senha pode ser descoberta pelo repositório ou por tentativa simples, dando acesso à rede da parede. Prioridade crítica. **Correção.** Ler a credencial de configuração protegida, gerar uma senha forte por instalação e trocá-la antes das demonstrações.

5.4 R04 CONFIANÇA NO RESULTADO DO APLICATIVO

Diagnóstico. O endpoint de sessões aceita nível, conclusão, pontuação e tempos informados pelo cliente (*backend/src/presentation/http/sessions.controller.ts:18-25* e *backend/src/presentation/http/dto/record-session.dto.ts:17-28*). **Risco e impacto.** Uma requisição fabricada pode conceder XP, liberar níveis e alterar estatísticas sem que a partida tenha ocorrido. Prioridade crítica. **Correção.** Criar a sessão no backend, aceitar apenas eventos válidos, encerrar uma única vez e calcular a recompensa no servidor.

5.5 R05 IDENTIFICADOR GLOBAL DE SESSÃO NA PAREDE

Diagnóstico. O ESP32 guarda uma única sessão e qualquer cliente pode substituí-la pelo comando session (*backend/firmware/smash_wall/wall_link.cpp:22 e 38-50*). **Risco e impacto.** Outro aparelho pode misturar impactos entre jogadores ou associá-los ao treino errado. Prioridade alta. **Correção.** Vincular a sessão ao cliente pareado, rejeitar mudanças durante o treino e limpar o valor ao desconectar.

5.6 R06 AUSÊNCIA DE LIMITAÇÃO DE LOGIN E CADASTRO

Diagnóstico. As rotas de sign-in e sign-up não possuem limite por IP, conta ou intervalo (*backend/src/presentation/http/auth.controller.ts:8-25*). **Risco e impacto.** Tentativas automatizadas podem testar senhas, criar várias contas e consumir recursos da API e do banco. Prioridade alta. **Correção.** Aplicar limitação progressiva, registrar falhas sem senhas e bloquear temporariamente padrões abusivos.

5.7 R07 FALTA DE LIMITES EXPLÍCITOS NO ESP32

Diagnóstico. O protocolo valida o JSON, mas não fixa quantidade de clientes, tamanho dos quadros ou comandos por segundo (*backend/firmware/smash_wall/wall_protocol.cpp:65-67*). **Risco e impacto.** Uma rajada de mensagens pode consumir memória e processamento, atrasando sensores ou reiniciando a parede. Prioridade alta. **Correção.** Aceitar somente os clientes necessários, limitar tamanho e frequência, encerrar ociosos e testar tráfego anormal.

5.8 R08 EXPOSIÇÃO AMPLA DA API NA REDE

Diagnóstico. O backend permite CORS sem lista de origens e escuta em 0.0.0.0 (*backend/src/main.ts:19 e 24*). **Risco e impacto.** Em rede compartilhada, outros dispositivos e páginas não previstas podem alcançar as rotas, ampliando a superfície de ataque. Prioridade média. **Correção.** Definir origens por ambiente, restringir host e firewall e expor somente o proxy HTTPS.

5.9 R09 CREDENCIAIS PADRÃO DO BANCO

Diagnóstico. Sem variáveis de ambiente, a configuração usa usuário e senha smash, também descritos no guia local (*backend/src/infrastructure/database/database.config.ts:13-17*). **Risco e impacto.** Se o PostgreSQL aceitar conexões externas ou a configuração for copiada, os dados poderão ser lidos ou alterados. Prioridade alta. **Correção.** Exigir senha exclusiva fora do ambiente local, usar privilégio mínimo e limitar o banco aos endereços necessários.

5.10 R10 SESSÃO JWT LONGA E SEM REVOGAÇÃO

Diagnóstico. O token dura sete dias e o logout o remove apenas do aplicativo (*backend/src/infrastructure/infrastructure.module.ts:65* e *backend/.env.example:8*).

Risco e impacto. Um token capturado continua válido até expirar, mesmo depois de o usuário sair. Prioridade média. **Correção.** Reduzir a duração do acesso, adotar renovação controlada e permitir revogação por sessão ou identificador jti.

5.11 R11 ALERTAS EM DEPENDÊNCIAS

Diagnóstico. O npm audit de produção registrou 9 alertas no backend e 18 moderados no frontend (*backend/package.json* e *frontend/package.json*). **Risco e impacto.** Uma falha alcançável pode afetar principalmente a disponibilidade; alertas de recursos não usados ainda precisam ser confirmados. Prioridade alta até a triagem. **Correção.** Aplicar correções compatíveis, testar atualizações principais separadamente e justificar apenas os alertas comprovadamente não alcançáveis.

5.12 R12 MONITORAMENTO DE SEGURANÇA LIMITADO

Diagnóstico. A API não mantém trilha específica de logins falhos, tokens inválidos, abuso ou alterações de sessão (registro apenas em *backend/firmware/smash_wall/wall_link.cpp:30 e 35*). **Risco e impacto.** A equipe pode detectar um incidente tarde e ter pouca informação para entender sua origem. Prioridade média. **Correção.** Criar logs estruturados sem senha ou token completo e revisar os eventos após testes e demonstrações.

5.13 R13 CONTROLES PARA O MQTT FUTURO

Diagnóstico. A integração prevista ainda não possui broker, autenticação, TLS ou regras por tópico no código atual (sem arquivo correspondente em *backend/src* nem em *backend/firmware/smash_wall*). **Risco e impacto.** Se o MQTT for aberto, clientes da rede poderão publicar impactos falsos, ler eventos ou inundar o backend. Prioridade alta para a fase de integração. **Correção.** Antes de implementar, exigir TLS, conta própria por parede, bloqueio de anônimos, limites e ACL apenas para *smash/{wall_id}/impacto*.

6 PRIORIDADES DE CORREÇÃO

As recomendações anteriores podem ser executadas nesta ordem, de acordo com o momento do projeto:

- Antes da próxima demonstração: trocar a senha do SoftAP, isolar a rede da parede e aceitar somente o celular utilizado no teste.
- Antes de usar rede compartilhada: proteger a API com HTTPS, restringir CORS e firewall e substituir a credencial padrão do banco.
- Antes de ranking ou comparação entre usuários: validar sessões no backend e impedir resultados fabricados ou encerramentos repetidos.
- Na evolução do sistema: limitar autenticação e tráfego do ESP32, melhorar logs, revisar dependências e usar MQTT somente com TLS e ACL por parede.

As correções classificadas como críticas e altas serão implementadas no código e documentadas na Entrega 2 da disciplina, que trata da mitigação aplicada. Esta primeira entrega delimita o diagnóstico, as evidências e a ordem de execução.

7 CONCLUSÃO

O Smash já aplica controles importantes: bcrypt, JWT, validação de dados, consultas parametrizadas, transações e filtragem dos eventos da parede. Essa base é adequada ao protótipo, mas não elimina os riscos das conexões entre os componentes.

As prioridades são retirar credenciais previsíveis, isolar a rede, impedir clientes não autorizados, proteger a API com HTTPS e validar no servidor o resultado das sessões. Com essas ações, o projeto preserva a integridade dos treinos e atende ao objetivo de identificar, documentar e tratar vulnerabilidades do sistema e do código.